Intelligent Multi-Class Natural Language Text Sentiment Classifier Using TF-IDF and Machine Learning

Muhammad Zain Tariq
BS Computer Engineering
Email: zaintariq967@gmail.com

***Abstract*—This paper presents a multi-class sentiment classifier using Natural Language Processing (NLP) and machine learning. Text is preprocessed using stop-word removal and lemmatization before TF-IDF feature extraction. Logistic Regression, Multinomial Naive Bayes, and Linear SVM are evaluated. A Gradio dashboard enables interactive sentiment prediction and visualization.**

***Index Terms*—Sentiment Analysis, NLP, TF-IDF, Linear SVM, Logistic Regression, Naive Bayes, Gradio**

I. Introduction

Sentiment analysis automatically identifies opinions expressed in text. This project classifies text into Positive, Negative and Neutral sentiments using classical machine learning models.

II. Objectives

- Build a multi-class sentiment classifier.
- Compare Logistic Regression, Naive Bayes and Linear SVM.
- Develop an interactive Gradio dashboard.

III. Dataset

Twitter Entity Sentiment Analysis dataset from Kaggle was used. Remove the Irrelevant class if only three classes are required.

IV. Methodology

Workflow: Dataset → Cleaning → Lemmatization → TF-IDF → Train/Test Split → Model Training → Evaluation → Gradio UI.

A. Text Preprocessing

- Lowercase conversion
- URL removal
- Punctuation removal
- Stop-word removal
- Lemmatization

B. TF-IDF

$$TFIDF(t,d) = TF(t,d) \times \log \frac{N}{DF(t)}$$

V. Models

A. Logistic Regression

Baseline linear classifier.

B. Multinomial Naive Bayes

Probabilistic classifier:

$$P(c|d) \propto P(c) \prod_i P(w_i|c)$$

C. Linear SVM

Optimization:

$$\min \frac{1}{2}||w||^2 + C \sum_i \xi_i$$

VI. Implementation

A. Training Code

```
vectorizer=TfidfVectorizer(
 max_features=15000,
 ngram_range=(1,2),
 min_df=2)

X=vectorizer.fit_transform(df["Clean_Text"])

svm=LinearSVC()
svm.fit(X_train, y_train)
```

B. Prediction Code

```
text=["I■love■this■internship"]
clean=[clean_text(t) for t in text]
vector=vectorizer.transform(clean)
prediction=svm.predict(vector)
```

VII. Experimental Results

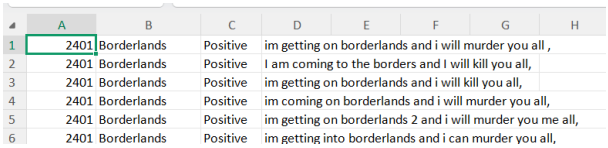
VIII. Figures

IX. Discussion

Linear SVM generally achieved the highest weighted F1-score because of its effectiveness on sparse high-dimensional TF-IDF features.

TABLE I
MODEL COMPARISON

Model	Weighted F1
Linear SVM	0.77
Logistic Regression	0.75
Naive Bayes	0.71



	A	B	C	D	E	F	G	H
1	2401	Borderlands	Positive	im getting on borderlands and i will murder you all ,				
2	2401	Borderlands	Positive	I am coming to the borders and I will kill you all,				
3	2401	Borderlands	Positive	im getting on borderlands and i will kill you all,				
4	2401	Borderlands	Positive	im coming on borderlands and i will murder you all,				
5	2401	Borderlands	Positive	im getting on borderlands 2 and i will murder you me all,				
6	2401	Borderlands	Positive	im getting into borderlands and i can murder you all,				

Fig. 1. Dataset Preview

X. FUTURE WORK

- Hyperparameter tuning
- Transformer models (BERT)
- Multilingual sentiment analysis
- Cloud deployment

XI. CONCLUSION

The proposed system successfully classifies text into Positive, Negative and Neutral classes. Classical NLP combined with Linear SVM provides an efficient and accurate solution while the Gradio dashboard enables interactive predictions.

REFERENCES

[1] Scikit-Learn Documentation.
[2] NLTK Documentation.
[3] Gradio Documentation.
[4] Twitter Entity Sentiment Analysis Dataset, Kaggle.

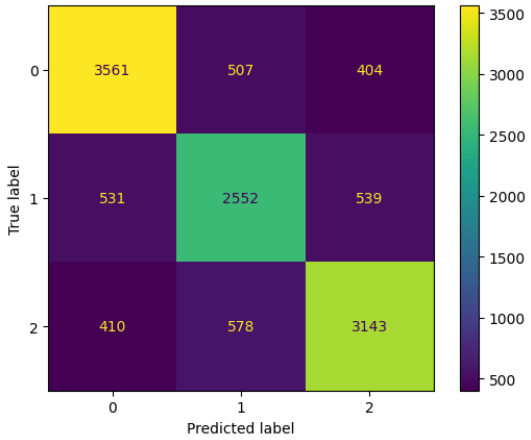


Fig. 2. Confusion Matrix of the Linear SVM Classifier

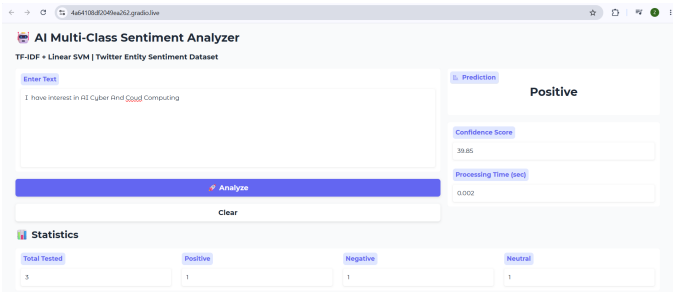


Fig. 3. Home Screen of the Gradio-based Sentiment Analysis Dashboard



Fig. 4. Sentiment Prediction Showing the Predicted Class and Confidence Score

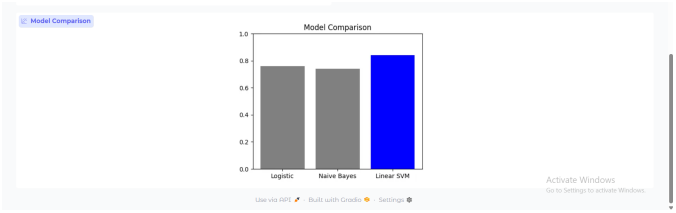


Fig. 5. Gradio Dashboard Displaying Sentiment Statistics and Prediction History